#1 Method `mergeClusters` takes two shards, extending two adjacent cells, a set of margin points, and a disjoint set on the list of clusters. We suppose that clusters have already been re-indexed globally, and that `clustersSet` is a *facade* emitting a pair of clusters to merge.
#2 For each margin point in the intersection between the two shards (we assume `marginPoints` is an instance of a class handling also these kind of operations – abstracting away the complexity that is not relevant to this method).
#3 Check if `p` is a core point in at least one of the two shards (also, we assume the shard objects created by mappers handle this kind of methods). If it is not, we can ignore it.
#4 Check if `p` is classified as noise, as an outlier, in the first shard.
#5 If it is so, it will be in a cluster in `shard2`, and we only need to remove it from the list of outliers in `shard1`.
#6 Same thing if it is an outlier in `shard2` (also, remember it can only be an outlier in one of the shards, as it's a core point in at least one)
#7 If p is not an outlier in either shard, it means it is assigned to a cluster in both: hence we need to merge those clusters.
#8 The relevant information we changed is all in the set of clusters and the two shards. Assuming on we pass these structures by value (at least the shards), we can return them at the end of the method.

The implementation goes through all points in the margin region between the two shards (use figure 13.12 as a reference), and checks if any of these points is a core point in at least one of the shards; then we just need to make sure it's not a noise point in the other shard (handled as an edge